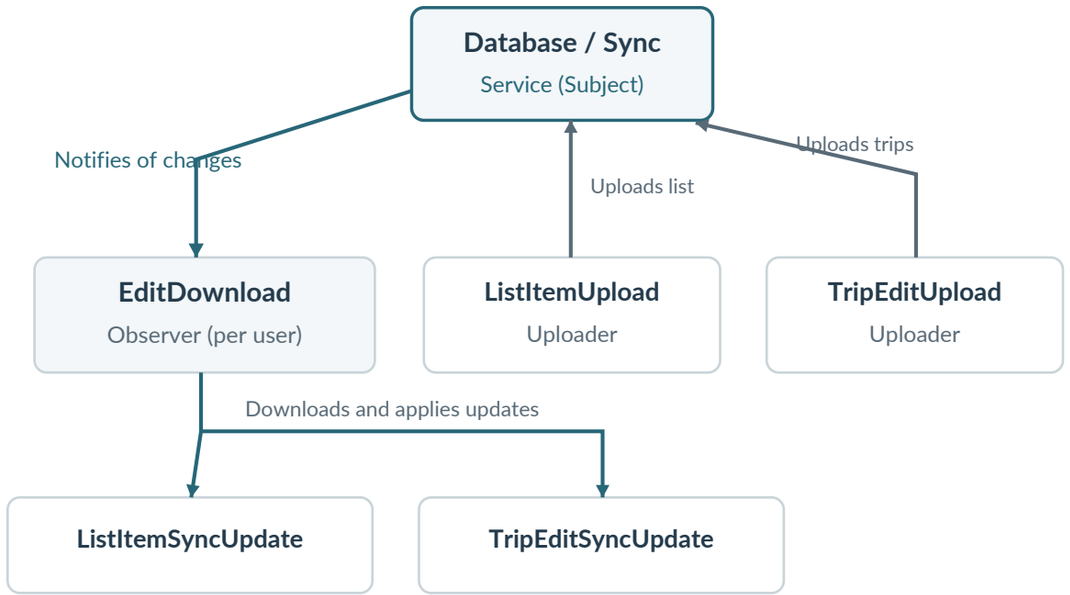
Synchronization design sketch



TE 4 Design Decisions and Trade-offs
For backend, we considered both express and hono as backend frameworks, but went with hono because it worked better in a serverless environment especially with cloudflare because it had less overhead processing and took advantage of the work cloudflare already did for us.
For auth, we considered Supabase Auth, BetterAuth, Passport, and implementing our own OAuth but decided on BetterAuth because we were using a database other than supabase and BetterAuth plugged in to our backend framework and ORM really easily without having to do much extra work, and our systems are more secure my using a tested and trusted framework rather than implementing OAuth ourselves.

TE4 Design Patterns

Pattern: Observer (Planned)

Problem it solves in our application:

Every user in a grocery group needs their local copy of the grocery list and grocery trip information updated when another user makes changes. The Observer pattern allows users to automatically receive updates whenever shared information changes.

Classes, modules, components, or services involved:

- ListItemUpload
- TripEditUpload
- EditDownload
- ListItemSyncUpdate
- TripEditSyncUpdate
- Database / Synchronization Service

Why this pattern is a better choice than placing all of the logic in one large component or module:

The individual upload, download, and update components can be maintained separately without modifying the entire system. New fields or features can be synchronized by adding or modifying the appropriate components. The Observer pattern also allows different components to respond to changes independently, helping all users maintain an up-to-date local copy of the shared grocery list and trip information.

This is planned. We plan to implement it by:

We plan to implement this by creating a class for each component listed and using JSON files to transfer the information that needs to be synced. When a user makes a change, ListItemUpload or TripEditUpload will upload the changes to the database. The synchronization service will notify each user's EditDownload observer, which will download the updated data and use ListItemSyncUpdate or TripEditSyncUpdate to modify the user's local copy.